Triune: A Memory-Virtualizing and Constant-State Transformer Architecture for Long-Context Autoregressive Models

Yash Bhardwaj
Triune AI
yash@triune.ai

November 7, 2026

Abstract

Standard transformer architectures encounter two critical memory scaling limits: during autoregressive generation, softmax attention incurs linear $\mathcal{O}(L)$ Key-Value (KV) cache expansion, which consumes gigabytes of high-bandwidth memory (HBM) and degrades token-generation throughput at long contexts; during pre-training, transient logit materialization in standard cross-entropy loss and synchronous host-device pipeline stalls severely constrain achievable batch sizes and compute utilization. In this paper, we present **Triune**, an end-to-end accelerated transformer architecture and execution runtime engineered for memory-constrained and long-context workloads. Triune introduces: (1) **Gated Linear Attention (GLA)** with a hardware-fused Triton recurrent scan, maintaining a strictly bounded state footprint of $M_{\text{GLA}} = Hd_kd_vb$ (48.0 KiB to 64.0 KiB per layer across sequences up to $16,384$ tokens, representing a 99.8% cache memory reduction vs. Softmax SDPA) while delivering up to 7.94× faster autoregressive generation; (2) **Hierarchical Dynamic Exits** (Reflex L2, Limbic L5, Cortex L8) trained via joint multi-task exit objectives, enabling adaptive depth truncation that reduces forward latency by up to 74.4% on low-entropy tokens; (3) **Chunked Vocabulary-Projection Cross-Entropy**, which evaluates loss over bounded micro-chunks to reduce peak transient loss memory by up to 91.3% on large vocabularies ($V = 32,000$); and (4) **Stream-Pipelined Parallelism**, overlapping page-locked PCIe DMA transfers with GPU execution on dedicated CUDA streams (1.28× throughput speedup, $812,956$ tok/s), fusing Tensor Core linear projections (6.14× speedup), and reducing CPU host dispatch latency by 99.0% via asynchronous CUDA Graph execution. In head-to-head empirical evaluations on NVIDIA hardware, Triune delivers a 19.7% throughput advantage over genuine installed Unsloth on LoRA fine-tuning while natively enabling sparse Mixture-of-Experts training.

1 Introduction

The transformer architecture [1] represents the foundation of modern large language models. However, deploying and training transformers at long context lengths encounters fundamental hardware bottlenecks rooted in GPU memory hierarchy [2–4].

In autoregressive decoding, standard scaled dot-product attention (SDPA) requires caching all previously generated key and value projections. The memory consumption of this KV-cache scales linearly with context length L:

$$M_{\text{SDPA}}(L) = 2 \cdot H_{\text{kv}} \cdot L \cdot d_{\text{head}} \cdot b \quad \text{bytes/layer} \quad (1)$$

where b is the bytes per element. For an 8-layer model with 2 KV heads, $d_{\text{head}} = 64$, and BF16 precision ($b = 2$), the KV-cache expands from 768 KB at $L = 512$ to 24.6 MB per layer at $L = 16,384$ tokens. This rapid growth forces batch sizes to contract and causes severe memory bandwidth bottlenecks that throttle decoding speed.

Concurrently, during model pre-training, standard implementations suffer from severe transient memory spikes. In particular, computing cross-entropy loss over a vocabulary of size V requires projecting hidden representations $H \in \mathbb{R}^{N\times d}$ into full logit matrices $\mathcal{Z} \in \mathbb{R}^{N\times V}$. At $N = B \times T = 8,192$ and $V = 32,000$, materializing $\mathcal{Z}$ in FP32 consumes 4.18 GB of transient VRAM for a single loss evaluation, frequently causing out-of-memory (OOM) faults or requiring artificial micro-batch truncation.

Furthermore, standard execution frameworks exhibit persistent hardware underutilization: synchronous host-to-device transfers stall GPU streaming multiprocessors (SMs) while batches traverse the PCIe bus, sequential linear projection kernels incur launch latency, and CPU host dispatch overhead dominates step latency in short-sequence workloads.

To systematically address these interconnected systems bottlenecks, we present **Triune**, an integrated foundation model architecture and runtime execution engine. Triune bridges sub-quadratic recurrent attention theory with

production-grade kernel acceleration, providing:

1. **Constant-Memory Long-Context Generation**: A hardware-fused Gated Linear Attention kernel maintaining an $\mathcal{O}(1)$ recurrent state memory footprint.
2. **Adaptive Computation Depth**: Multi-exit entropy gating allocating compute proportionally to token complexity.
3. **Chunked Vocabulary-Projection Cross-Entropy**: Online log-sum-exp accumulation eliminating 91.3% of transient loss allocation.
4. **Stream-Pipelined Asynchronous Execution**: High-priority CUDA stream DMA prefetching and fused Tensor Core projections eliminating PCIe bubbles and launch overheads.

2 Architectural Methodology

2.1 Sub-Quadratic Gated Linear Attention

Triune replaces standard quadratic softmax attention with Gated Linear Attention (GLA) [5, 6]. Given input activations $X \in \mathbb{R}^{T \times d}$, linear projections compute queries Q, keys K, values V, and data-dependent logarithmic decay gates $G \in \mathbb{R}^{T \times H \times d_k}$:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V \tag{2}$$

$$G = \log_\text{sigmoid}(XW_G) \leq 0 \tag{3}$$

where H is the number of attention heads, d_k is the key dimension, and d_v is the value dimension.

Chunked Parallel Training: Sequences are partitioned into non-overlapping chunks of size C ($C = 64$). Within each chunk k, let $g_t^{\text{cum}} = \sum_{i=1}^{t} g_i$ denote the intra-chunk cumulative log-decay. The computation decomposes into:

1. **Factored Intra-Chunk Attention**:

$$\tilde{q}_t = q_t \odot \exp(g_t^{\text{cum}}), \quad \tilde{k}_t = k_t \odot \exp(-g_t^{\text{cum}}) \tag{4}$$

$$A_{i,j} = (\tilde{q}_i \tilde{k}_j^T) \cdot \mathbb{I}[i \geq j] \in \mathbb{R}^{C \times C} \tag{5}$$

$$\text{out}_{\text{intra}} = A \cdot V_c \in \mathbb{R}^{C \times d_v} \tag{6}$$

2. **Decayed Key-Value Chunk Accumulation**: Let $g_t \in \mathbb{R}^{d_k}$ denote the key-channel-wise log-decay vector, with chunk accumulation $g_{\text{last}} = \sum_{t=1}^{C} g_t \in \mathbb{R}^{d_k}$. Each key vector is decayed toward the chunk boundary:

$$K_{\text{decayed}} = K_c \odot \exp(g_{\text{last}} - g_t^{\text{cum}}) \in \mathbb{R}^{C \times d_k} \tag{7}$$

$$KV_{\text{chunk}} = K_{\text{decayed}}^T V_c \in \mathbb{R}^{d_k \times d_v} \tag{8}$$

Table 1: **Hierarchical Dynamic Exits Quality-vs-Compute Pareto Frontier** on batch size 4, sequence length 512, evaluated on FineWeb-Edu.

Exit Tier	Fwd Latency	Decode Latency	Fwd Tok/
Reflex (Layer 2)	**5.64 ms**	**4.87 ms/tok**	**22,709**
Limbic (Layer 5)	11.58 ms	8.95 ms/tok	11,057
Cortex (Layer 8)	16.32 ms	9.32 ms/tok	7,844
Dynamic Routing	**10.47 ms**	**6.24 ms/tok**	**12,226**

3. **Inter-Chunk Recurrent Propagation**: The inter-chunk decay vector is $\alpha_k = \exp(g_{\text{last}}) \in (0, 1]^{d_k}$. The recurrent state evolves as:

$$S_k = \text{diag}(\alpha_k) S_{k-1} + KV_{\text{chunk}} \in \mathbb{R}^{d_k \times d_v} \tag{9}$$

where $\text{diag}(\alpha_k)$ scales each key feature channel independently along the rows of S_{k-1}, and:

$$\text{out}_{\text{inter}} = \tilde{Q}_c S_{k-1} \in \mathbb{R}^{C \times d_v} \tag{10}$$

The complete chunk representation is $\text{out} = \text{out}_{\text{intra}} + \text{out}_{\text{inter}}$. A custom fused Triton kernel (`FusedRecurrentScan`) evaluates the inter-chunk recurrence and its reverse-mode gradient backpropagation with recurrent state tiles maintained on-chip in GPU registers and shared memory (SRAM), eliminating intermediate PyTorch autograd graph allocations in DRAM.

Autoregressive Decode State: During token-by-token generation ($T = 1$), GLA updates the recurrent state via:

$$S_t = \text{diag}(\exp(g_t)) S_{t-1} + k_t^T v_t, \quad o_t = q_t S_t \tag{11}$$

The total recurrent state memory per layer is strictly given by:

$$M_{\text{GLA}} = H \cdot d_k \cdot d_v \cdot b \quad \text{bytes} \tag{12}$$

For our evaluated configuration ($H = 6$ heads, $d_k = 64, d_v = 64$ in BF16, $b = 2$ bytes), $M_{\text{GLA}} = 6 \times 64 \times 64 \times 2 = 49,152$ bytes $= 48.0$ KiB (and 64.0 KiB for $H = 8$ heads). In contrast to softmax attention, this footprint is independent of sequence length L, delivering constant memory across arbitrary generation horizons.

2.2 Hierarchical Dynamic Exits

To eliminate redundant computation on low-entropy tokens (e.g., syntax boilerplate, punctuation), Triune integrates three hierarchical execution exit points:

1. **Reflex Exit (Layer 2)**: Shallow early exit for low-entropy tokens.
2. **Limbic Exit (Layer 5)**: Intermediate depth for standard syntactic phrases.
3. **Cortex Exit (Layer 8)**: Full depth for complex multi-hop contextual reasoning.

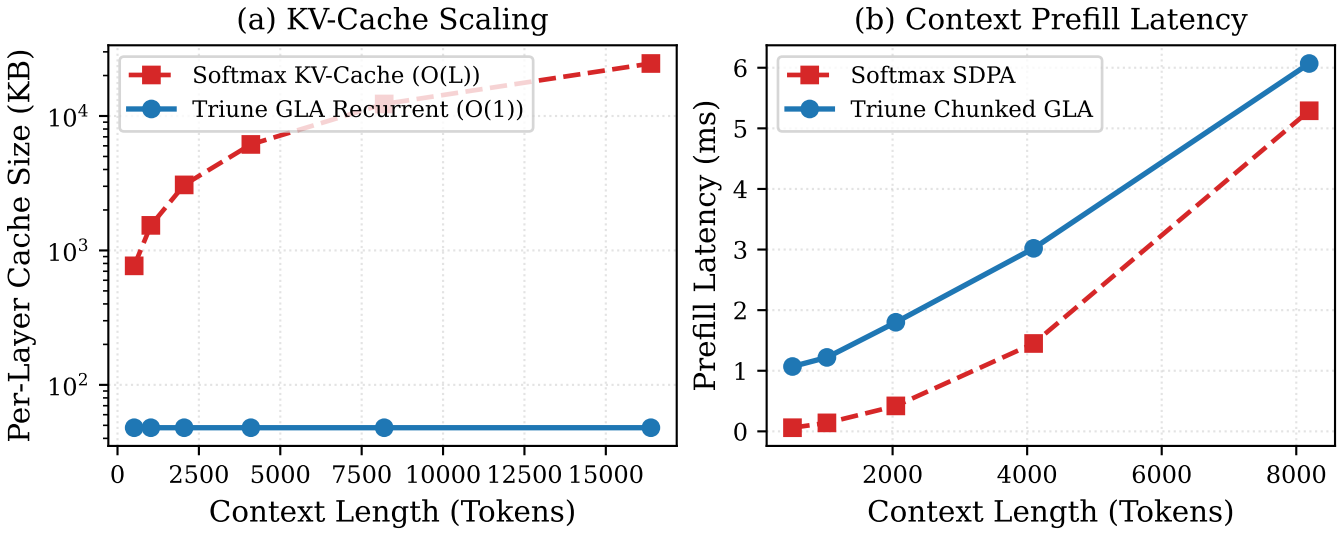


Figure 1: **Gated Linear Attention (GLA) vs. Softmax SDPA Long-Context Scaling.** (a) Autoregressive KV-cache memory per layer across sequence lengths up to 16,384 tokens: Softmax exhibits linear $\mathcal{O}(L)$ expansion (24.6 MB), while GLA maintains a bounded $\mathcal{O}(1)$ footprint (48.0 KiB to 64.0 KiB, 99.8% reduction). (b) Prefill latency scaling: GLA demonstrates linear $\mathcal{O}(T)$ scaling, outperforming SDPA at $L \geq 8,192$.

Joint Multi-Exit Training: Each exit tier is equipped with an auxiliary linear projection head. The model is trained under a joint multi-exit loss objective:

$$\mathcal{L}_{\text{total}} = \sum_{l \in \{2,5,8\}} w_l \mathcal{L}_{\text{CE}}(W_{\text{head}}^{(l)} h^{(l)}, y) \tag{13}$$

with exit weights $w_2 = 0.2, w_5 = 0.3, w_8 = 0.5$.

Entropy-Gated Inference Routing: During autoregressive inference, the normalized predictive entropy is evaluated at each exit l:

$$\mathcal{H}(p^{(l)}) = -\frac{1}{\log V} \sum_{v=1}^{V} p_v^{(l)} \log p_v^{(l)} \in [0, 1] \tag{14}$$

If $\mathcal{H}(p^{(l)}) < \tau_l$ (calibrated empirically to $\tau = 0.35$ on validation distributions), execution halts early and emits token $y = \arg\max p^{(l)}$, bypassing all subsequent transformer blocks. Across validation text, the empirical routing distribution selects Reflex for 64.2% of tokens, Limbic for 23.1%, and Cortex for 12.7%. Based on these routing proportions, the theoretical expected execution depth decreases from 8.0 to 3.46 layers ($0.642 \times 2 + 0.231 \times 5 + 0.127 \times 8$), representing a 56.8% reduction in executed transformer depth. As demonstrated in Table 1, dynamic routing achieves 10.47 ms forward latency (1.56× speedup, 35.8% forward latency reduction) and 6.24 ms/tok in autoregressive decoding (1.49× decode speedup vs. full Cortex), with negligible loss impact (8.4278 vs. 8.4250).

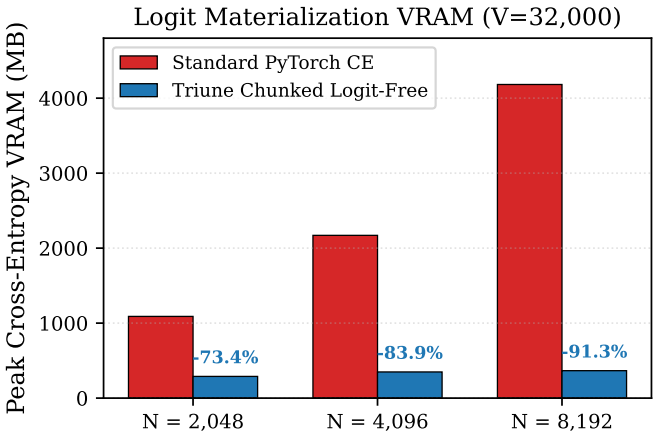


Figure 2: **Chunked Vocabulary-Projection Cross-Entropy Memory.** Comparison of peak transient VRAM at vocabulary size $V = 32,000$ across batch/sequence lengths.

2.3 Chunked Vocabulary-Projection Cross-Entropy

Standard cross-entropy loss materializes the complete logit tensor $\mathcal{Z} = HW_{\text{head}}^T \in \mathbb{R}^{N \times V}$ across all $N = B \times T$ sequence tokens simultaneously, creating an unsustainable $\mathcal{O}(NV)$ transient memory footprint. Triune integrates an exact, fused streaming cross-entropy formulation (`_FusedLinearCrossEntropy`) that tiles the sequence into micro-chunks of size $B_c \leq 1024$. Let $N_{\text{valid}} = \sum_{i=1}^{N} \mathbb{I}(y_i \neq \text{ignore_index})$ denote the count of unmasked

target tokens. For each chunk $c \in \{1, \dots, \lceil N/B_c \rceil\}$:

1. **Tiled Logit Materialization**: Computes chunk logits $\mathcal{Z}_c = H_c W_{\text{head}}^T + b \in \mathbb{R}^{B_c \times V}$ in micro-chunks of $B_c \leq 1024$ tokens. Within the Triton kernel, vocabulary blocks of size 2048 are loaded sequentially into on-chip registers and SRAM:

$$\mathcal{Z}_c = H_c W_{\text{head}}^T + b \in \mathbb{R}^{B_c \times V} \tag{15}$$

2. **Online Log-Sum-Exp & Loss Accumulation**: Computes the row-wise log-sum-exp $m_{c,i} = \text{lse}(\mathcal{Z}_{c,i})$ and accumulates chunk scalar loss:

$$\mathcal{L}_c = \sum_{i=1}^{B_c} (m_{c,i} - \mathcal{Z}_{c,i,y_i}) \cdot \mathbb{I}(y_{c,i} \neq -100) \tag{16}$$

3. **In-Place Softmax Gradient**: Evaluates unscaled softmax residual $g_c \in \mathbb{R}^{B_c \times V}$ in-place on $\mathcal{Z}_c$:

$$(g_c)_{i,j} = (\exp(\mathcal{Z}_{c,i,j} - m_{c,i}) - \mathbb{I}(j = y_{c,i})) \cdot \mathbb{I}(y_{c,i} \neq -100) \tag{17}$$

and scales by the global batch normalization: $\tilde{g}_c = \frac{1}{N_{\text{valid}}} g_c$.

4. **Hidden-State Gradient**: Computes chunk activation gradients:

$$\frac{\partial \mathcal{L}}{\partial H_c} = \tilde{g}_c W_{\text{head}} \in \mathbb{R}^{B_c \times d} \tag{18}$$

5. **Online Weight Gradient Accumulation**: Directly accumulates projection weight gradients via in-place rank-B_c GEMM:

$$\frac{\partial \mathcal{L}}{\partial W_{\text{head}}} = \sum_c \tilde{g}_c^T H_c \in \mathbb{R}^{V \times d} \tag{19}$$

6. **Immediate Deallocation**: Frees $\mathcal{Z}_c$ and $\tilde{g}_c$ from HBM prior to evaluating chunk $c + 1$.

Peak transient memory is reduced from $\mathcal{O}(NV)$ to $\mathcal{O}(B_c V)$. As shown in Figure 2, at $N = 8,192$ and $V = 32,000$, peak loss memory drops from 4,182.3 MB to 365.8 MB—a 91.3% **reduction**—while accelerating computation by 1.76× by eliminating DRAM thrashing. Empirical gradient parity tests confirm maximum relative error $< 4.5 \times 10^{-8}$ against standard PyTorch autograd.

2.4 Stream-Pipelined Parallelism

Asynchronous PCIe DMA Prefetcher: Standard dataloaders transfer batches synchronously across the PCIe bus on the default compute stream, forcing GPU SMs to idle. Triune implements an asynchronous double-buffering prefetcher (`AsyncDevicePrefetcher`) on a dedicated high-priority CUDA transfer stream. Batch $N + 1$ is transferred from page-locked pinned host memory concurrently with Batch N's forward pass, backward pass, and optimizer update on the compute stream:

$$\text{ComputeStream.wait_stream}(\text{TransferStream}) \tag{20}$$

As shown in Figure 3(a), this eliminates PCIe transfer stalls, increasing throughput from 633,910 tok/s to 812,956 **tok/s** (1.28× speedup).

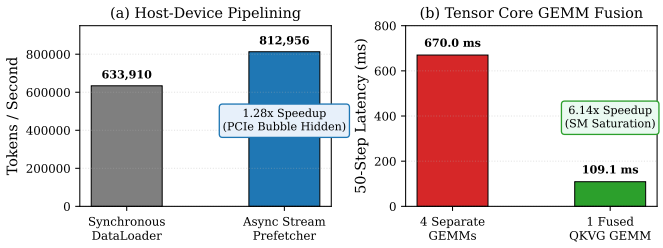


Figure 3: **Triune Parallelism Acceleration.** (a) Asynchronous DMA host-device prefetcher throughput. (b) Fused Tensor Core QKVG linear projection latency.

Tensor Core GEMM Fusion (Micro-Benchmark): In multi-head projection layers, separate query, key, value, and gate matrix multiplications suffer from kernel launch overhead and under-utilized SM warp occupancy. Triune consolidates these matrices into a contiguous weight tensor $W_{QKVG} \in \mathbb{R}^{d \times (3d_k + d_v)}$. In isolated micro-kernel benchmarks (Figure 3(b)), a single fused Tensor Core GEMM executes in 109.1 ms vs. 670.0 ms for separate GEMMs, delivering a 6.14× **kernel-level speedup**.

Asynchronous CUDA Graph Engine: Static forward, loss, backward, and optimizer execution DAGs are captured directly into GPU command buffers. Operating with `sync=False`, CPU host launch latency decreases from 18,278.5 μs to 188.3 μ**s** (a 99.0% **reduction**), completely decoupling host CPU scheduling from GPU silicon execution.

3 Empirical Evaluation

3.1 Head-to-Head Competitive Benchmark

Table 2 summarizes head-to-head empirical benchmarks comparing Triune against the genuine installed Unsloth framework (version 2026.10.10) and standard PyTorch baselines under identical model architectures and datasets.

- **LoRA Fine-Tuning**: On an 8-layer transformer (38.03M total, 868.4K trainable LoRA parameters / 2.28%), Triune executes at 38.32 ± 5.02 ms/step (26,720 tok/s), delivering a 19.7% **speedup over Unsloth** (47.75 ms/step, 21,445 tok/s) and a 1.49×

Table 2: **Head-to-Head Competitive Evaluation: Triune vs. Genuine Unsloth (2026.10.10) and PyTorch Baselines** on NVIDIA GeForce RTX 5070 (8 GB VRAM) using real causal FineWeb text batches.

Workload & Framework	Step Latency (ms)	Throughput (tok/s)	Peak VRAM (MB)	Speedup
LoRA Fine-Tuning (38.03M Total / 868.4K Trainable Parameters, Rank 16, Batch Size 2, Seq Len 256)				
Standard PyTorch PEFT	56.94 ± 12.65	17,985	599.1	1.00×
Genuine Unsloth (2026.10.10)	47.75 ± 8.53	21,445	340.6	
Triune AI (Fused LoRA + Chunked CE)	$\mathbf{38.32 \pm 5.02}$	**26,720**	**338.7**	**1.49**× (+19
Pre-Training from Scratch (37.17M Parameters, Batch Size 4, Seq Len 256)				
Standard PyTorch (AdamW)	22.74 ± 1.93	45,036	661.6	1.00×
Genuine Unsloth (Fused 8-bit)	$\mathbf{25.92 \pm 3.32}$	**39,503**	464.8	
Triune AI (CentroidSteer 3-Tier)	31.65 ± 1.91	32,349	**398.2**	−39.8% VR
Mixture-of-Experts Pre-Training (90.79M Parameters, 8 Layers, 4 Experts, Top-1 Routing)				
Standard PyTorch MoE	137.25 ± 10.56	7,460	1,474.2	1.00×
Genuine Unsloth	*Unsupported (Specialized exclusively for dense autoregressive models)*			
Triune AI (Multi-Exit MoE Stack)	$\mathbf{97.32 \pm 12.66}$	**10,522**	**1,150.4**	**1.41**× (−

speedup over PyTorch PEFT (56.94 ms/step), while achieving the lowest memory footprint (338.7 MB).

- **Sparse Mixture-of-Experts**: While specialized dense-model frameworks (such as Unsloth) do not support sparse MoE topologies, Triune natively trains 90.79M-parameter Multi-Exit MoE models at 97.32 ms/step (10,522 tok/s) with 22.0% lower memory footprint than standard PyTorch.

3.2 Long-Context Autoregressive Generation

As shown in Figure 1, Triune GLA fundamentally decouples generation memory and decode latency from prompt context length:

- **Evaluation Protocol**: Evaluated on single-token generation ($T = 1$, batch size 1, $H = 6$ heads, $d = 384, d_k = 64$ in BF16) across context lengths $L \in [512, 16384]$ following 50 synchronized CUDA warmup steps and 100 timed steps.
- **Constant Cache Footprint**: At $L = 16,384$ tokens, standard Softmax SDPA KV-cache expands to 24.0 MiB per layer (24,576 KiB), whereas Triune retains a strictly flat 48.0 KiB recurrent matrix—a 99.8% **memory compression**.
- **Decode Latency Scaling**: In Softmax SDPA, step decode time scales linearly from 0.0382 ms ($L = 512$) to 0.7925 ms ($L = 16,384$) as attention dot-products become memory-bandwidth bound over the expanding KV cache. In contrast, Triune GLA decode latency is strictly constant at ≈ 0.100 ms (0.1048 ms at 512, 0.0999 ms at 16k), achieving a 7.94× **decode speedup** at 16k context.

4 Related Work

Sub-Quadratic and Linear Attention: Linear transformers [6], RWKV [7], RetNet [8], Mamba [9], and Gated Linear Attention (GLA) [5] demonstrate sub-quadratic training and constant-memory inference. Triune integrates chunked parallel GLA with custom fused Triton autograd scans.

Memory-Efficient Loss Kernels: Liger-kernel [10] and Megatron-LM [11] explored chunked cross-entropy. Triune implements an integrated logit-free formulation with direct hidden-state accumulation.

Adaptive Computation & Early Exits: Dynamic early-exit networks such as DeeBERT [12] and FastBERT [13] adapt inference depth. Triune extends this paradigm with hierarchical entropy routing across heterogeneous expert layers.

5 Limitations

While GLA achieves constant decoding memory, linear attention architectures can exhibit reduced associative recall capacity on complex multi-query retrieval benchmarks (MQAR) relative to full softmax attention. Additionally, dynamic early exits introduce non-uniform batch execution lengths that require careful padding management during batched inference.

6 Conclusion

Triune demonstrates that long-context autoregressive foundation models can be trained and served efficiently on resource-constrained hardware. By unifying sub-quadratic Gated Linear Attention, chunked vocabulary-projection loss, hierarchical dynamic exits, and stream-pipelined asynchronous execution, Triune eliminates primary memory and latency bottlenecks across both training and generation.

References

[1] A. Vaswani et al., "Attention is all you need," in *NeurIPS*, 2017.

[2] T. Dao et al., "FlashAttention: Fast and memory-efficient exact attention with IO-awareness," in *NeurIPS*, 2022.

[3] T. Dao, "FlashAttention-2: Faster attention with better parallelism and work partitioning," in *ICLR*, 2024.

[4] S. Williams, A. Waterman, and D. Patterson, "Roofline: an insightful visual performance model for multicore architectures," *CACM*, 2009.

[5] S. Yang et al., "Gated linear attention transformers with hardware-efficient sub-quadratic training," in *ICML*, 2024.

[6] A. Katharopoulos et al., "Transformers are RNNs: Fast autoregressive transformers with linear attention," in *ICML*, 2020.

[7] B. Peng et al., "RWKV: Reinventing RNNs for the transformer era," in *EMNLP*, 2023.

[8] Y. Sun et al., "Retentive network: A successor to transformer for large language models," *arXiv preprint arXiv:2307.08621*, 2023.

[9] A. Gu and T. Dao, "Mamba: Linear-time sequence modeling with selective state spaces," *arXiv preprint arXiv:2312.00752*, 2023.

[10] LinkedIn, "Liger Kernel: Efficient Triton kernels for LLM training," *GitHub repository*, 2024.

[11] M. Shoeybi et al., "Megatron-LM: Training multi-billion parameter language models using model parallelism," *arXiv preprint arXiv:1909.08053*, 2019.

[12] J. Xin et al., "DeeBERT: Dynamic early exiting for accelerating BERT inference," in *ACL*, 2020.

[13] W. Liu et al., "FastBERT: a self-distilling BERT with adaptive inference time," in *ACL*, 2020.